Master's Thesis Proposal

Preliminary Title

Lightweight AI for Detecting Irregular Behavior in GPU Node Telemetry

Deepak Budha — MSc Data Science, University of Göttingen / GWDG — deepak.budha@stud.uni-goettingen.de
Supervisor: Michael Bidollahkhani **1st Reviewer:** Prof. Dr. Julian Kunkel **Date:** 21 August 2026 **Revised:** 19 October 2026

Aims of the Thesis

This thesis aims to systematically characterize the performance-efficiency trade-off for lightweight anomaly detection models running on a companion node in HPC systems: which model family, and at what compression level, provides the optimal balance between detection quality and resource efficiency for detecting structural GPU anomalies in production telemetry.

Why worth conducting. GPU-equipped nodes are central to modern HPC and AI workloads. A significant class of GPU failures—device detachments at the driver or interconnect level ("fallen off bus" events)—exhibits little or no numeric precursor in conventional telemetry (temperature, power, utilization, clocks) [Bidollahkhani et al., 2026a]. The dominant observable signal is instead *structural*: GPU metrics disappear while other metrics of the same node, e.g. CPU load, are still reported. This pattern separates a GPU failure from a node crash or network failure, where all reporting stops. It is a property of the failure mode itself, not a shortcoming of any detection method, and this thesis therefore treats such meta-metrics (missing metrics, scrape indicators) as features. The detector runs on a separate companion node, independent of the monitored GPU node, so that a failing node does not take its own detector down with it. It must be lightweight, because its CPU, memory and latency footprint is what the evaluation profiles. This is why model compression matters. Here, compression means reducing model size and compute: quantization (lower numeric precision) and pruning (removing weights) for neural networks, and tree-specific simplification for Isolation Forest.

Why novel and needed. No existing work systematically compares multiple model families (classical ML, deep learning, compressed variants) for structural anomaly detection in GPU telemetry, nor characterizes model architecture, compression level, detection quality, and resource efficiency as a unified trade-off surface. Compression-focused edge-AI work in adjacent domains frequently reports model-level metrics without physical on-device measurement [Sun et al., 2026]—a gap this thesis addresses.

Research Questions

- **RQ1:** How do Isolation Forest and a Dense Autoencoder compare on GWDG GPU telemetry in detection quality (precision, recall, F1, lead time) and resource footprint (model size, latency, memory) before compression? Which meta-metrics (e.g. missing metrics, scrape indicators) contribute most to detection, and does this generalize across nodes or is it node-specific (per-node vs. one shared model)? *(LSTM Autoencoder added as a third, optional candidate if time permits.) Rationale:* establishes an empirical baseline so later compression is applied to genuinely evaluated models, not an arbitrarily preselected one.
- **RQ2:** How does compression affect each baseline candidate (quantization and/or pruning for neural networks, tree-specific simplification for Isolation Forest), and can the compressed models meet companion-node-representative resource profiles without unacceptable loss of detection quality? *Rationale:* asked of every candidate, not one preselected model, since uncompressed accuracy does not predict compression robustness.

- **RQ3:** What is the resulting trade-off curve between detection performance and resource efficiency across models and compression levels—and does the pre-compression ranking still hold after compression? *Rationale:* turns feasibility (RQ2) into a quantified curve; the thesis's core empirical contribution.
- **RQ4:** Does detecting on a companion node, independent of the monitored GPU node, offer a practical advantage over centralized detection when the monitoring pipeline itself is degrading? *Rationale:* connects compression results back to the motivating problem—GPU detachments degrade the very pipeline a centralized detector depends on.

Material and Methods

Data. GWDG GPU Node Telemetry Dataset [Bidollahkhani et al., 2026b]: sanitized production telemetry from 7 GPU nodes, aligned to an operator-curated incident catalog (Jan 2025–Feb 2026). 21 telemetry windows (16 incident + 5 healthy reference), 11-column schema, ∼15.2M rows across GPU (DCGM), node/OS, and monitoring-pipeline (Prometheus scrape) planes. Incident catalog: 69 incidents across 8 categories (GPU error/problem: 31; GPU fell off bus: 24; GPU unknown: 5; GPU lost: 3; remaining categories: 6).

Intended usage environment. The detector runs on a separate companion node, not on the monitored GPU node. It reads the per-node telemetry (GPU, node/OS and monitoring-pipeline metrics) of the node it watches, does not use the GPU (the component that may fail), and does not depend on the monitored node staying healthy. Physical deployment is out of scope: the models are evaluated offline on the recorded telemetry, and their resource use (CPU, RAM, inference latency, model size) is profiled in a reproducible setup representative of the companion-node environment. Figure 1 illustrates the intended setting.

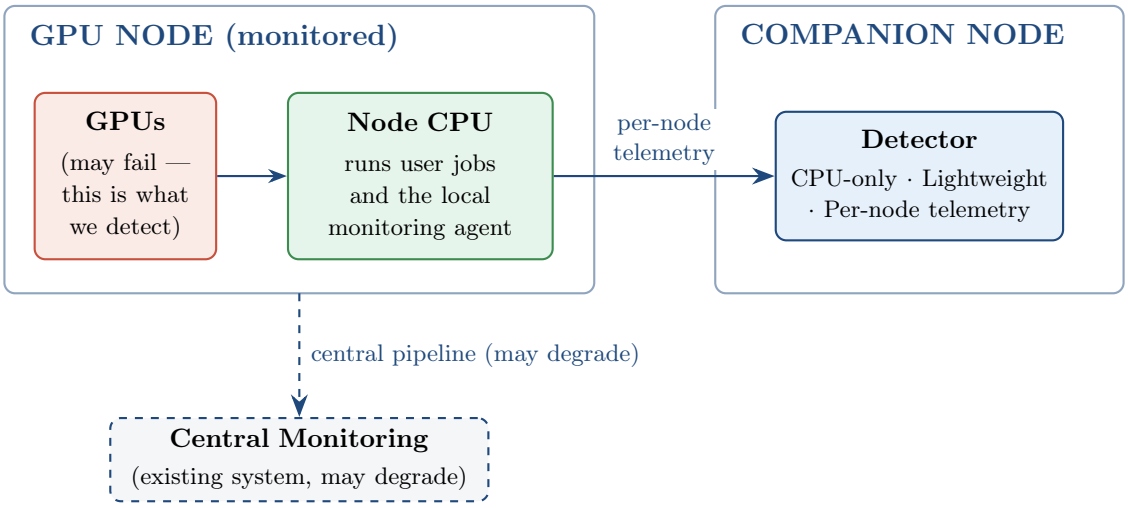


Figure 1: Intended setting. The detector runs on a separate companion node and reads the per-node telemetry of the monitored GPU node, independently of the central monitoring pipeline.

Feature engineering and sampling. Three feature planes—GPU (temperature, power, utilization, clocks), observability (scrape success rate, duration, metric availability), and OS (CPU, memory, network I/O)—aggregated into fixed windows (60-minute length, 10-minute stride, to be confirmed with supervisor) with per-window statistics (mean, max, min, std, slope). Feature relevance is assessed by comparing detection quality across feature planes. Train/validation split is chronological, not random; models train only on healthy reference windows (Q1–Q4 2025, Q1 2026), incident windows withheld entirely. Both per-node and pooled (shared) models are compared, since per-node models are known to outperform shared models in HPC telemetry [Borghesi et al., 2019a].

Models. *Compulsory:* Isolation Forest (unsupervised, minimal resource footprint) and Dense Au-

toencoder (semi-supervised, trained on healthy data only, reconstruction-error detection). *Optional stretch goal:* LSTM Autoencoder, pursued only if time permits.

Procedure. *Phase 1 (RQ1):* train both compulsory models on healthy data; measure precision, recall, F1, and lead time, alongside model size, inference latency, training time, and memory. *Phase 2 (RQ2–RQ3):* apply quantization (FP32/FP16/INT8) and pruning (0/50/75/90% sparsity), individually and combined, to the neural candidates, and tree-specific simplification (e.g. fewer trees, reduced depth) to Isolation Forest, so that *every* baseline candidate is compressed with the techniques that apply to its family—not only the stronger uncompressed performer; record the same quality and resource metrics per configuration; construct trade-off curves across models and compression levels. *Phase 3 (RQ4):* compare centralized (full telemetry stream) vs. companion-node (per-node telemetry only) detection under a stress test that withholds progressively larger fractions of telemetry (10–90%) to simulate pipeline degradation.

Evaluation. Primary evaluation is profiling-based (model size, CPU, RAM, inference latency)—sufficient on its own to answer RQ2–RQ3, following established practice in the compression literature [Sun et al., 2026]. The profiling is reproducible and representative of the companion-node environment, i.e. measured against a documented CPU and memory budget. Physical deployment is out of scope. Software: Python/Pandas/NumPy; scikit-learn (Isolation Forest); TensorFlow/Keras (autoencoders); TensorFlow Lite (quantization).

Expected Results

1. Autoencoders will outperform Isolation Forest on detection quality, but at higher resource cost. Autoencoders have reported 90–95% accuracy on HPC telemetry [Borghesi et al., 2019a], and deep learning methods are generally reported as more accurate while requiring only normal samples for training [Lu et al., 2026]; Isolation Forest is expected to have the smaller footprint. *Visualization:* bar chart comparing F1 and model size, uncompressed, both models.

2. Compression will reduce model size substantially with modest performance loss, but the two models are not expected to be equally compressible. Compression literature reports large size reductions with small accuracy cost from combined pruning and quantization [Sun et al., 2026], and pruning has demonstrated an 8.15× latency reduction on Raspberry Pi in an unrelated domain [Lee et al., 2023]; the exact magnitude for this thesis's models and dataset is to be established empirically rather than assumed. The Dense Autoencoder is expected to be more amenable to compression than Isolation Forest, which has no weights to quantize or prune, given its neural-network structure. *Visualization:* trade-off curves (F1 vs. model size) per model per compression level.

3. Companion-node detection is expected to preserve signal availability better than centralized detection as pipeline degradation increases. Because GPU detachment failures manifest as monitoring-pipeline degradation [Bidollahkhani et al., 2026a], a threshold telemetry-drop rate is expected beyond which companion-node detection outperforms centralized detection. *Visualization:* line chart of detection quality vs. telemetry drop rate (0–90%) for both detection modes.

References

Bidollahkhani, M., Nordsiek, F., & Kunkel, J. M. (2026a). When GPUs Fail Quietly: Observability-Aware Early Warning Beyond Numeric Telemetry. *arXiv preprint arXiv:2603.28781*.

Bidollahkhani, M., Nordsiek, F., & Kunkel, J. M. (2026b). GWDG GPU Node Telemetry Dataset for Observability-Aware Early Warning of GPU Detachment Failures (2025–2026). *Zenodo*. https://doi.org/10.5281/zenodo.19052367

Borghesi, A., Libri, A., Benini, L., & Bartolini, A. (2019a). Online Anomaly Detection in HPC Systems. In *2019 IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS). arXiv preprint arXiv:1902.08447*.

Lee, D., Lee, E., & Hwang, Y. (2023). Lossless Reconstruction of Convolutional Neural Network for Channel-Based Network Pruning. *Sensors*, 23(4), 2102. https://doi.org/10.3390/s23042102

Lu, Y., Zhang, W., Yang, L., Dai, Y., Wu, H., Wu, Z., Tian, H., Li, Y., Chen, J., Liu, C., & Dong, Y. (2026). A survey of anomaly detection in HPC systems using machine learning. *CCF Transactions on High Performance Computing*, 8, 352–376. https://doi.org/10.1007/s42514-025-00266-7

Sun, Y., Qin, Y., Chen, W., Zhao, W., & Sun, H. (2026). EPC-TinyAD: An Energy- and Privacy-Aware Compressed TinyML Framework for Reliable Industrial Anomaly Detection on Resource-Constrained Edge Devices. *Electronics*, 15(13), 2879. https://doi.org/10.3390/electronics15132879